Machine Learning Applications

Python Programming Fundamentals

Course Overview

This introductory Machine Learning course covers fundamental concepts, algorithms, and techniques. Today's session focuses on **Python — the most commonly used language of data science and ML**.

What We'll Cover Today

- Primitive data types & variables
- Collections: List, Tuple, Dictionary
- Control flow: if/elif/else, loops
- Functions and scope
- Slicing & indexing
- Classes, objects, and inheritance

Why Python for ML?

- **Readable syntax** — close to pseudocode, great for non-CS backgrounds
- **Rich ecosystem** — NumPy, Pandas, Scikit-learn, TensorFlow
- **Community** — largest data science community & resources
- **Rapid prototyping** — interpreted, dynamically typed

Part 1

Variables & Primitive Data Types

Variables & Dynamic Typing

> Python is **dynamically typed** — you don't declare a variable's type. The interpreter infers it at runtime. A single variable can hold different types over its lifetime.

Key Concepts

- **No type declarations** — just assign a value
- **type()** function reveals the current type
- Python uses **references** — variables point to objects in memory

💡 Compare with Java/C# where you write `int x = 5;` — Python simply uses `x = 5`. This lowers the barrier to entry but requires careful attention to types at runtime.

variables.py ▶ Run

```
# Try changing these values and
x = 42
print(f"x = {x}, type: {type(x).

x = 3.14
print(f"x = {x}, type: {type(x).

x = "Hello, NUS!"
print(f"x = {x}, type: {type(x).

x = True
print(f"x = {x}, type: {type(x).
```

OUTPUT

Primitive Data Types

> **Primitive** data types are the building blocks — predefined by the language. Python's core primitives are **int**, **float**, **str**, **bool**, and the special **NoneType**.

Type	Example	Use Case
`int`	`42`, `-7`	Counting, indexing
`float`	`3.14`, `-0.5`	Measurements, probabilities
`str`	`"hello"`, `'hi'`	Text data, labels
`bool`	`True`, `False`	Conditions, flags
`NoneType`	`None`	Absence of value

💡 Unlike C/Java, Python integers have **arbitrary precision** — no overflow! Try computing `2 ** 100`.

primitives.py ▶ Run

```
# Experiment with type conversic
a = 10
b = 3

print("Integer division:", a //
print("Float division:  ", a / b
print("Modulus:         ", a % b

# Type conversion
c = str(a)
print(f"\nstr({a}) = '{c}' (type

# Boolean context — what's truth
```

OUTPUT

String Operations

> Strings in Python are **immutable sequences** of characters. They support rich operations critical for text processing in ML — tokenization, cleaning, and formatting.

Key String Features

- Defined with single quotes `'...'` or double quotes `"..."`
- **f-strings** (Python 3.6+): `f"Hello {name}"` — the modern way to format
- Common methods: `.upper()`, `.lower()`, `.split()`, `.strip()`, `.replace()`
- Strings are **iterable** — you can loop through each character

💡 In ML, you'll use string operations extensively during **text preprocessing** — cleaning datasets, extracting features from text.

strings.py ▶ Run

```
name = "Machine Learning"

# f-string formatting (modern Py
print(f"Course: {name}")
print(f"Upper:  {name.upper()}")
print(f"Length: {len(name)}")

# Splitting — useful for tokeniz
words = name.split(" ")
print(f"Words:  {words}")

# String is a sequence
print(f"First char: '{name[0]}'"
```

OUTPUT

Part 2

Collection Data Types — List, Tuple, Dictionary

Lists — Mutable Ordered Collections

A **List** is a **mutable, ordered** collection that can hold elements of different types. Lists are your go-to general-purpose container in Python — similar to arrays in other languages, but far more flexible.

Common Operations

- `append(x)` — add item to end
- `insert(i, x)` — insert at position
- `remove(x)` — remove first occurrence
- `pop(i)` — remove & return item at index
- `sort()` — sort in place
- `len(list)` — get length

💡 In ML, you'll rarely use raw lists for numerical data (NumPy arrays are faster). But lists are essential for managing labels, file paths, and configuration.

lists.py ▶ Run

```
# Creating a list
students = ["Alice", "Bob", "Cha
print("Original:", students)

# Append and insert
students.append("Diana")
students.insert(1, "Eve")
print("After add:", students)

# Access by index (0-based!)
print(f"First: {students[0]}")
print(f"Last:  {students[-1]}")
```

OUTPUT

Tuples — Immutable Ordered Collections

A **Tuple** is like a list, but **immutable** — once created, its elements cannot be changed. This makes tuples safer for data that shouldn't be modified, and they can serve as dictionary keys.

List vs Tuple

Feature	List	Tuple
Syntax	`[1, 2, 3]`	`(1, 2, 3)`
Mutable?	Yes	No
Dict key?	No	Yes
Performance	Slower	Faster

💡 **When to use tuples:** function return values (returning multiple items), dictionary keys, and any data that should remain constant (e.g., RGB color values, coordinates).

tuples.py ▶ Run

```
# Creating tuples
point = (3, 4)
rgb = (255, 128, 0)

print(f"Point: {point}")
print(f"Color: {rgb}")

# Access elements
print(f"x = {point[0]}, y = {poi

# Tuple unpacking — very Pythoni
x, y = point
print(f"Unpacked: x={x}, y={y}")
```

OUTPUT

Dictionaries — Key-Value Stores

A **Dictionary** maps **keys** to **values** — like a real dictionary maps words to definitions. Keys must be immutable (strings, numbers, tuples). Dictionaries are **mutable** and extremely fast for lookups.

Common Operations

- `d[key]` — access value (raises KeyError if missing)
- `d.get(key, default)` — safe access with fallback
- `d.keys()`, `d.values()`, `d.items()`
- `key in d` — check if key exists
- `del d[key]` — remove a key-value pair

💡 Dictionaries are fundamental in ML: model hyperparameters, JSON data, feature maps, and configuration are all stored as dictionaries.

dictionaries.py ▶ Run

```
# ML hyperparameters as a dict
config = {
    "learning_rate": 0.001,
    "epochs": 50,
    "batch_size": 32,
    "optimizer": "adam"
}

print("Config:", config)
print(f"LR: {config['learning_ra

# Safe access with .get()
print(f"Dropout: {config.get('dr
```

OUTPUT

Part 3

Control Flow — Conditionals & Loops

Conditionals — if / elif / else

Python uses **colons** to start code blocks and **indentation** (typically 4 spaces) to define scope — no curly braces needed. This enforces readable code by design.

Syntax Rules

- Each branch ends with a **colon :**
- The body is **indented** (4 spaces by convention)
- `elif` = "else if" — can have multiple
- `else` is optional and catches everything remaining

Comparison Operators

- == equal, != not equal
- <, >, <=, >=
- and, or, not (logical)
- `is` — identity check (same object)

conditionals.py ▶ Run

```
# Try changing the score!
score = 78

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 60:
    grade = "D"
else:
    grade = "F"
```

OUTPUT

For Loops & Iteration

> Python's **for** loop iterates over any **iterable** (list, tuple, string, range, ...). The `range(start, stop, step)` function generates integer sequences — `start` defaults to 0, `step` defaults to 1.

Iteration Patterns

- `for x in list:` — iterate by value
- `for i in range(n):` — iterate by index
- `for i, val in enumerate(list):` — both index & value
- `for k, v in dict.items():` — dict key-value

List Comprehensions

A concise one-line syntax for creating lists:
`[expression for item in iterable if condition]`

> 💡 `enumerate()` is a Pythonic way to avoid manual index tracking — use it instead of `range(len(x))`.

loops.py ▶ Run

```
fruits = ["apple", "banana", "ch

# ❌ Beginner pattern: manual ir
# Verbose, error-prone, less rea
print("Manual index (avoid this)
for i in range(len(fruits)):
    print(f"  {i}: {fruits[i]}")

# ✅ Pythonic: enumerate() gives
print("\nWith enumerate (prefer
for i, fruit in enumerate(fruits
    print(f"  {i}: {fruit}")

# range() examples
```

OUTPUT

While Loops & Loop Control

A **while** loop repeats as long as its condition is True. Use **break** to exit early, and **continue** to skip to the next iteration.

Loop Control Keywords

- **`break`** — immediately exit the loop
- **`continue`** — skip to the next iteration
- **`pass`** — do nothing (placeholder)

💡 **Caution:** while loops can run forever if the condition never becomes False. Always ensure the loop variable is updated, or use `break` as a safety exit. For counting loops, prefer `for`.

🧠 **Quick Think**

What does this print?

while_loops.py ▶ Run

```
# Simple countdown
count = 5
while count > 0:
    print(f"T-minus {count}...")
    count -= 1
print("Liftoff! 🚀\n")

# break & continue example
print("Finding first even number
n = 1
while True:
    n += 1
    if n % 2 != 0:
        continue   # skip odd nu
```

```
    x += 1
    if x == 2:
        continue
    print(x)
```

1 2 3

1 3

0 1 3

```
        break       # exit the l

# Practical: simple convergence
print("\nSimulated training:")
loss = 1.0
epoch = 0
while loss > 0.01:
    loss *= 0.7  # decay
    epoch += 1
    print(f"  Epoch {epoch}: los
print(f"Converged at epoch {epoc
```

Part 4

Functions — Reusable Code Blocks

Defining & Calling Functions

> Functions are defined with **def**, don't require return type annotations, and must be **defined before they are called**. Python supports **default arguments**, **keyword arguments**, and **multiple return values**.

Function Features

- **No return type needed:** unlike Java (`int add(int a, int b)`), Python just uses `def add(a, b):` — the return type is inferred at runtime
- **Default args:** `def f(x, y=10):`
- **Keyword args:** `f(y=20, x=5)`
- **Multiple returns:** `return a, b` (returns a tuple)
- **Docstrings:** triple-quoted string right after `def`
- **Lambda:** `lambda x: x**2` — anonymous one-liner

💡 **Best practice:** write docstrings for all your functions. Others (and future-you) will thank you. Use `help(func)` to view them.

functions.py ▶ Run

```
def greet(name, greeting="Hello"
    """Return a personalized gre
    return f"{greeting}, {name}!

# Different ways to call
print(greet("Alice"))
print(greet("Bob", "Hi"))
print(greet(greeting="Hey", name

# Multiple return values
def min_max(numbers):
    """Return both min and max."
    return min(numbers), max(num
```

OUTPUT

Slicing & Negative Indexing

> **Slicing** extracts subsequences using `[start:stop:step]`. **Negative indexing** counts from the end: `-1` is the last element. These work on lists, strings, tuples — any sequence.

Slicing Rules

- `start` is **inclusive**, `stop` is **exclusive**
- Omit `start` → defaults to 0
- Omit `stop` → defaults to end
- `[::-1]` — reverse the entire sequence

Useful: rfind()

- `str.rfind(sub)` — find **last** occurrence of sub
- Returns the index position, or **-1** if not found
- Syntax: `rfind(substring, start, end)`

slicing.py ▶ Run

```
x = [10, 20, 30, 40, 50]

print("Full list:", x)
print("x[1:3]  =", x[1:3])    #
print("x[:3]   =", x[:3])     #
print("x[2:]   =", x[2:])     #
print("x[::2]  =", x[::2])    #
print("x[::-1] =", x[::-1])   #

# Negative indexing
print(f"\nLast item:  x[-1] = {x
print(f"2nd last:   x[-2] = {x[-
print(f"Last three: x[-3:] = {x[
```

```
Index:    0   1   2   3   4
Values:  [a,  b,  c,  d,  e]
Neg idx: -5  -4  -3  -2  -1
```

```
print(f"\n'{s}' reversed = '{s[:
print(f"First 3 chars: '{s[:3]}'

# Practical: extract file extens
# rfind() = find LAST occurrence
# Returns the index, or -1 if nc
filename = "model_weights.h5"
dot_pos = filename.rfind('.')
print(f"\nrfind('.') in '{filena
ext = filename[dot_pos:]
print(f"Extension: {ext}")

# rfind vs find
path = "data/train/images.tar.gz
```

OUTPUT

Part 5

Classes & Object-Oriented Programming

Classes & Objects

A **class** is a blueprint for creating objects. It bundles data (**attributes**) and behaviour (**methods**) together. The constructor is always `__init__`, and `self` refers to the current instance.

OOP Essentials

- `class ClassName:` — defines a class
- `__init__(self, ...)` — constructor method
- `self` — reference to the instance (like `this` in Java/C#)
- Attributes set via `self.attr = value`
- **No explicit access modifiers** — prefix with _ for "private" (convention only)

💡 In ML, you'll encounter classes heavily when using frameworks like PyTorch (defining custom `nn.Module` subclasses) and Scikit-learn (custom transformers and estimators).

classes.py ▶ Run

```
class Student:
    """Represents a postgraduate

    def __init__(self, name, prc
        self.name = name
        self.programme = program
        self.grades = []

    def add_grade(self, score):
        self.grades.append(score

    def average(self):
        if not self.grades:
```

OUTPUT

Class Inheritance

> **Inheritance** lets a child class extend a parent class, reusing and overriding behaviour. In Python, the parent class is specified in parentheses: `class Child(Parent):`. Use `super()` to call the parent's methods.

Inheritance Concepts

- **Code reuse** — child inherits all parent methods/attributes
- **Override** — child redefines a parent method
- **`super().__init__()`** — call parent constructor
- **`isinstance(obj, Class)`** — check type
- Python supports **multiple inheritance** (use carefully)

💡 In PyTorch, every neural network inherits from `nn.Module` and overrides the `forward()` method. Understanding inheritance is essential for building custom ML models.

inheritance.py ▶ Run

```
class Person:
    def __init__(self, name, yea
        self.name = name
        self.year_of_birth = yea

    def introduce(self):
        return f"I'm {self.name}

class Employee(Person):
    def __init__(self, name, yea
        super().__init__(name, y
        self.company = company
```

OUTPUT

Error Handling — try / except

> Robust code handles errors gracefully. Python's **try/except** mechanism catches exceptions at runtime, preventing your programme from crashing. This is essential when dealing with real-world data that can be messy and unpredictable.

Exception Handling Structure

- `try:` — code that might fail
- `except ErrorType:` — handle specific errors
- `except:` — catch any error (use sparingly)
- `finally:` — always runs (cleanup)
- `raise` — throw your own exceptions

Common Exceptions

- `TypeError` — wrong type for operation
- `ValueError` — right type, wrong value

exceptions.py ▶ Run

```
# Basic try/except
def safe_divide(a, b):
    try:
        result = a / b
        return result
    except ZeroDivisionError:
        print("⚠ Cannot divide b
        return None

print(safe_divide(10, 3))
print(safe_divide(10, 0))

# Handling multiple exception ty
```

- `KeyError` — missing dictionary key
- `IndexError` — list index out of range
- `FileNotFoundError` — file doesn't exist

```
        value = int(raw)
        return value * 2
    except ValueError:
        print(f"⚠ '{raw}' is not
        return None
    except TypeError:
        print(f"⚠ Expected strir
        return None

print(f"\nparse_data('42') = {pa
print(f"parse_data('abc') = {par
print(f"parse_data(None) = {pars

# finally — always executes
```

OUTPUT

Summary & Next Steps

> Today you've learned the Python foundations needed for the rest of this ML course. In the next sessions, we'll build on these concepts with **NumPy**, **Pandas**, and your first ML algorithms.

What We Covered

- Variables, dynamic typing, and primitive types
- Strings and their operations
- Collections: lists (mutable), tuples (immutable), dictionaries (key-value)
- Control flow: if/elif/else, for loops, while loops
- Functions: default args, lambdas, multiple returns
- Slicing and negative indexing
- Object-oriented programming: classes, inheritance
- Error handling with try/except

Practice Resources

- Kaggle — Learn Python (free, interactive)
- Learn Python by Example
- Official Python Tutorial
- Python Functions (deep dive)
- Lambda Functions (deep dive)

💡 **Recommended practice:** Pick 3–5 exercises from the resources above. Focus on list comprehensions, dictionary manipulation, and writing small functions — these are the patterns you'll use most in ML code.

Thank You

Questions & Discussion

Machine Learning Applications · NUS Institute of Systems Science